

D-Bus调研

吴西飞

2014年8月19日



目录

- D-Bus简介
- D-Bus概念
- Process of Method-Call on D-Bus
- 实例代码
 - ❖ method call and signal emit: `libdbus`, `bus-glib`, `python`
 - ❖ 现有接口调用: `org.freedesktop.Notification`
 - ❖ 不同语言之间的dbus通信
- D-Bus相关工具
- dbus-binding: `node-dbus` `go-dbus`

D-Bus简介

- D-Bus is a system for interprocess communication (IPC). D-Bus is intended for potentially high-resolution same-machine IPC, not primarily for Internet IPC.
- low-level D-Bus have been heavily tested in the real world over several years, and are now "set in stone."
- IPC还有：Android使用Binder IPC、以及被D-Bus替代的CORBA，Bonobo和DCOP、xml格式的XML-RPC、Window上使用的DCOM (distributed COM)等。
- D-Bus建立在socket之上，与socket相比2.5x slower：marshaling、dbus-daemon、security等。

D-Bus简介

- D原先是代表桌面 “Desktop” 的意思。即用于桌面操作系统的通信总线。典型的桌面都会有多个应用程序在运行，而且，它们经常需要彼此进行通信。
- D-Bus is designed for two specific use cases:
 - ❖ 系统总线, system bus, system daemon (for events such as "new hardware device added" or "printer queue changed")
 - ❖ 会话总线, session bus, a per-user-login-session daemon (for general IPC needs among user applications).
- Security: the system bus and session bus are separate.

D-Bus简介

- **D-Bus has several layers:**
 - ❖ **libdbus allows two applications to connect to each other and exchange messages.**
 - ❖ **A message bus daemon can route messages from one application to zero or more other applications.**
 - ❖ **DBus bindings for particular application frameworks. For example, libdbus-glib and libdbus-qt. dbus-python, Perl, C++, php, .net, ruby, and so on.**
- **libdbus is not primarily designed for application authors to use, it is a basis for binding authors.**
- **DBus-binding simplify the details of D-Bus programming.**



D-Bus简介

- There are also some reimplementations of the D-Bus protocol for languages such as C#, Java, and Ruby.
- GDBus doesn't use the libdbus reference implementation. Instead, it relies on GIO streams as transport layer, and has its own implementation for the D-Bus connection setup and authentication.
- Gnome.org: GDBus is going to replace lib-dbus.
- D-Bus参考资料
 - ❖ <http://dbus.freedesktop.org/doc/dbus-tutorial.html>
 - ❖ <http://dbus.freedesktop.org/doc/dbus-specification.html>

D-Bus概念

■ Bus Names

- ❖ When each application connects to the bus daemon, the daemon immediately assigns it a name, called the unique connection name.
- ❖ Applications may ask to own additional well-known names. For example, you could write a specification to define a name called `com.mycompany.TextEditor`.

■ Queued Name Owner

- ❖ Each bus name has a primary owner; messages sent to the name go to the primary owner.
- ❖ If the primary owner releases the name, then the first secondary owner in the queue automatically becomes the new owner of the name.

D-Bus概念

■ Object Paths and Native Objects

- ❖ Native Object is a object defined in server side, such as `java.lang.Object`, `GObject`, `QObject`, `DBusMessage`, depending on the binding language used.
- ❖ The idea of an object path is that higher-level bindings can name native object instances, and allow remote applications to refer to them.

■ Methods and Signals (Object members)

- ❖ Methods are operations that can be invoked on an object, with optional input (aka arguments or "in parameters") and output (aka return values or "out parameters").
- ❖ Signals are broadcasts from the object to any interested observers of the object; signals may contain a data payload.

D-Bus概念

■ Interfaces

- ❖ Think of an interface as a named group of methods and signals, just as it is in GLib or Qt or Java. Interfaces define the type of an object instance.
 - ❖ Most bindings will map these interface names directly to the appropriate programming language construct, for example to Java interfaces or C++ pure virtual classes.
 - ❖ Each object supports one or more interfaces
- Interface is important for language binding. It is used for Mapping method calls to native APIs.

D-Bus概念

```
<?xml version="1.0" encoding="UTF-8" ?>
<node name="/">
  <interface name="org.ztb.Test">
    <method name="Add">
      <arg name="arg0" type="i"/>
      <arg name="arg1" type="i"/>
      <arg name="ret" type="i" direction="out"/>
    </method>
    <!-- Mark the signal as exported -->
    <signal name="HelloSignal"/>
  </interface>
</node>
```

dbus-binding-tool --prefix=test_object --mode=glib-server --output=example-signal-emitter-glue.h ./example-signal-emitter.xml

D-Bus概念

■ Messages

- ❖ Message is the basic unit travelled on bus daemon.
- ❖ Each message has a header, including fields, and a body, including arguments.
- ❖ There are 4 message types: Method call, Method return, Signal messages, Error messages.

■ marshaling and unmarshaling

- ❖ D-Bus has a type system, in which values of various types can be serialized into a sequence of bytes referred to as the wire format in a standard way.
- ❖ Converting a value from some other representation into the wire format is called marshaling
- ❖ converting it back from the wire format is unmarshaling.

D-Bus概念

■ Proxies

- ❖ **A proxy object is a convenient native object created to represent a remote object in another process.**

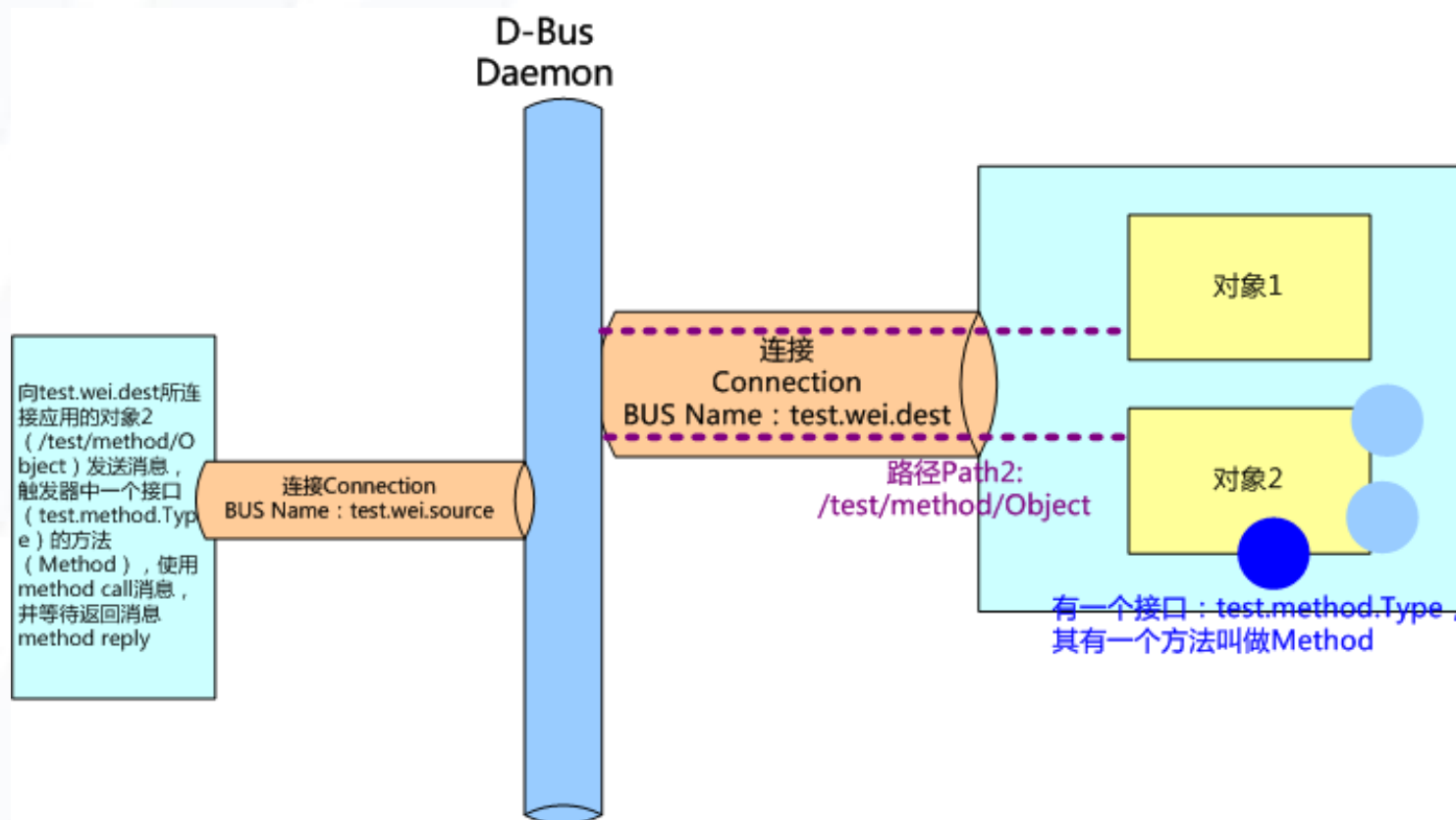
```
Proxy proxy = new Proxy(getBusConnection(), "/remote/object/path");
```

```
Object returnValue = proxy.MethodName(arg1, arg2);
```

■ How to Mapping method calls to native APIs

Process of Method-Call on D-Bus

- Method call on an object instance
 - ❖ Address -> [Bus Name] -> Path -> Interface -> Method



D-Bus相关工具

■ D-feet

- ❖ D-Feet is an easy to use D-Bus debugger.
- ❖ D-Feet can be used to inspect D-Bus interfaces and names of running programs and invoke methods on those interfaces.

■ dbus-monitor

■ dbus-send

- ❖ `dbus-send --session --type=method_call --print-reply --dest=org.ztb.Test /TestObj org.fmddlmyy.Test.Basic.Add int32:100 int32:999`

node-dbus

- It's unlikely that D-Bus is (or will be) a popular solution for IPC with Node developers, but it does allow us to integrate more tightly with desktop environments. (<http://dailyjs.com/2012/04/26/node-unix-ipc/>)
- The two most popular Node implementations are node-dbus They're built using C++ extensions that wrap around the libdbus APIs.
 - ❖ <https://www.npmjs.org/package/node-dbus>
 - ❖ <https://github.com/Shouqun/node-dbus>
 - ❖ <https://github.com/Motorola-Mobility/node-dbus>

node-dbus

- The node-dbus project is a simple light-weight NodeJS based wrapper over some libdbus api's which enables the developer to:
 - ❖ perform synchronous method-calls on a service provider
 - ❖ perform asynchronous method-calls on a service provider
 - ❖ send signals on the message bus
 - ❖ listen to signals propagated over the message bus
- Note that it is not intended to be a full-blown one-to-one mapping of the libdbus api. For that, you might want to look at node-libdbus which is relatively concrete.
 - ❖ https://github.com/agnat/node_libdbus

go.dbus

- go.dbus is a simple library that implements native Go client bindings for the D-Bus message bus system.
- Features:
 - ❖ Complete native implementation of the D-Bus message protocol
 - ❖ Go-like API (channels for signals / asynchronous method calls, Goroutine-safe connections)
 - ❖ Subpackages that help with the introspection / property interfaces
- `sudo env GOPATH=/home/cos/gocode go get github.com/guelfey/go.dbus`
- `env GOPATH=/home/cos/gocode go run list-names.go`

- 谢谢~
- Q&A

